

ECE 310

Project 3 Report

Harsh Shinde

April 24, 2026

Abstract

In this project, a serial Binary Coded Decimal (BCD) Arithmetic Logic Unit (ALU) was designed in Verilog to perform addition and subtraction on two 4-digit BCD operands. The system receives data serially on a 1-bit input, detects a fixed input header, stores the operands in parallel registers, performs the selected arithmetic operation, and then shifts the 5-digit BCD result out serially with an output header. The design uses a serial-in parallel-out (SIPO) input stage, a parallel BCD ALU core, and a parallel-in serial-out (PISO) output stage. Waveform simulation was used to verify addition, subtraction, continuous input handling, and reset behavior.

1 Introduction

The goal of this project was to implement a Serial BCD ALU that can add or subtract two 4-digit BCD values. The module receives a serial input `din`, where it begins with header `8'h67`, followed by a 1-bit operation select, then operand A , and then operand B . The design then computes either $A + B$ or $A - B$ and outputs the result serially on `result` with output header `8'hA5`.

Because the input and output are serial but the arithmetic is easier to perform in parallel, the design is divided into three main parts: an input SIPO register, a BCD ALU block, and an output PISO register. A synchronous active-high reset initializes the design to a known state. The design must also support continuous operation, meaning a new input can be shifted in without waiting for the previous output one to finish.

2 Background

BCD represents each decimal digit using 4 bits. Since only the decimal values 0 through 9 are valid, each 4-bit group corresponds to one decimal digit. In this project, each operand contains 4 BCD digits for a total width of 16 bits, and the result contains up to 5 BCD digits for a total width of 20 bits.

2.1 BCD Addition

BCD addition is similar to binary addition, but whenever a digit sum is greater than 9, a correction of 6 is added to produce a valid BCD digit and generate the carry to the next decimal position. For example,

$$6 + 7 = 13$$

so the BCD result becomes

$$0001\ 0011$$

which represents decimal 13.

2.2 BCD Subtraction

BCD subtraction is performed using 10's complement arithmetic. Each digit of operand B is first converted to its 9's complement, then 1 is added at the least significant digit. This allows subtraction to be implemented using addition logic. If a final carry is produced, it is discarded, which matches the expected decimal subtraction result.

3 Design

The design is divided into two modules: `BCD_ALU` and the top-level module `Project3`.

3.1 BCD_ALU Module

The `BCD_ALU` module performs either BCD addition or BCD subtraction on two 16-bit BCD inputs. The inputs are divided into four 4-bit digits. For subtraction, each digit of operand *B* is converted to its 9's complement, and the least significant digit receives an initial carry-in of 1 to complete the 10's complement operation.

The arithmetic is performed one BCD digit at a time starting from the least significant digit. After each digit addition, the intermediate sum is checked. If it is greater than 9, a correction value of 6 is added. This produces a valid BCD digit and the carry for the next stage. The final output is 20 bits wide so that a 5-digit result can be represented. For subtraction, the final carry is discarded and the most significant output digit is forced to 0.

3.2 Top-Level Module

The top-level module `Project3` connects the serial input stage, ALU core, and serial output stage.

Input Shift Register

A 41-bit shift register is used as a SIPO stage. As bits arrive on `din`, they are shifted into the register on each rising edge of the clock. The upper 8 bits of this register are continuously compared against the input header `8'h67`. When this pattern is detected in the correct position, the module treats the current register contents as a complete packet.

Packet Format

The input packet format is:

`8'h67    |    op    |    A[15:0]    |    B[15:0]`

where `op=0` selects addition and `op=1` selects subtraction.

The output packet format is:

`8'hA5    |    result[19:0]`

and is shifted out MSB first.

Sequence Detection

Once a valid input packet is detected, the top-level module latches the operation and operands into registers. At the same time, the input shift register is cleared. This creates a non-overlapping sequence detector so that a header pattern appearing inside operand data is not incorrectly treated as a new packet boundary.

Output Shift Register

After the ALU computes the result, the output packet `{8'hA5, alu_result}` is loaded into a 28-bit PISO register. This register shifts one bit out on each clock cycle through the `result` output. The output is sent MSB first.

4 Testing

The design was verified with a simple waveform-based testbench. The testbench applies reset once at the beginning, sends serial packets MSB first, and then leaves enough time to observe the output packet on the waveform. Only four major tests were used in order to keep the testbench simple while still covering the main behavior of the design.

Test	Operation	Inputs	Expected Result	Purpose
1	Add	$3627 + 1287$	04914	Basic addition example using the same format as the project description.
2	Subtract	$0637 - 0459$	00178	Verifies 10's complement subtraction and borrow discard behavior.
3	Continuous input	$9999 + 0001$, then $5000 - 5000$	10000, then 00000	Checks back-to-back packet handling and an overflow/carry-out case.
4	Reset recovery	Partial packet, reset, $5555 + 4444$	09999	Verifies that reset clears the system and allows correct operation afterward.

Table 1: Test cases used for simulation.

5 Result

5.1 Waveform Discussion

The waveform shows the four important signals: `clock`, `reset`, `din`, and `result`. At the start of simulation, `reset` is asserted to initialize all registers. After reset is released, the serial input packets are shifted into the module on `din`. Once a complete packet has been received and decoded, the corresponding output packet begins to appear on `result`.

The waveform confirms that the design accepts valid serial input, detects packet boundaries, computes the BCD result, and shifts the output back out serially. The visible bursts on `result` correspond to completed operations. The testbench also shows that the design can process back-to-back operations and can recover correctly after a reset in the middle of activity.

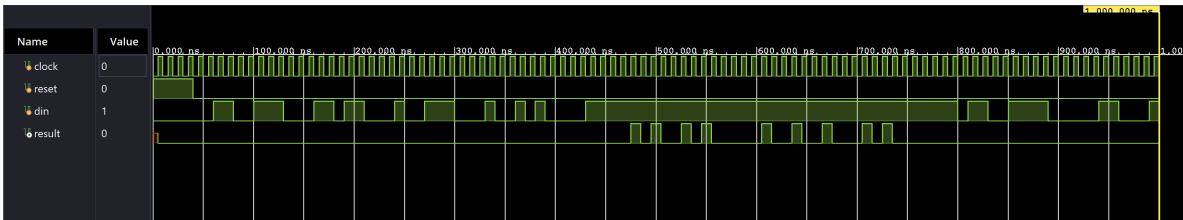


Figure 1: Simulation waveform showing reset, serial input `din`, and serial output `result`.

Simulation showed that the Serial BCD ALU performed the required operations correctly. Addition and subtraction both produced the expected BCD output packet format. The continuous-input test demonstrated that the design can accept a new input packet without waiting for the previous output packet to finish. The reset test showed that the system returns to a known state and resumes correct operation afterward.

The design also handles the required packet-based communication correctly by detecting the input header `8'h67` and generating the output header `8'hA5`. Clearing the input shift register after packet detection prevents false retriggering from operand data and supports non-overlapping sequence detection.

6 Conclusion

A serial BCD ALU was successfully designed and simulated in Verilog. The design accepts two 4-digit BCD operands and an operation bit through a serial input stream, performs either BCD addition

or BCD subtraction in parallel logic, and returns the 5-digit BCD result through a serial output stream. The overall structure of SIPO input, parallel ALU, and PISO output provided a simple and effective way to meet the project requirements. The simulation waveform verified the main functional requirements of the design, including addition, subtraction, continuous operation, and reset behavior. This project demonstrates how serial communication and parallel arithmetic can be combined in a clean top-level hardware design.

This report was compiled using L^AT_EX.